Smart Complaint Management System (SCMS)

Project Documentation

Institution: R. V. College of Engineering (RVCE) **Programme:** Master of Computer Applications (MCA), II Semester **Project Type:** Full-stack, AI-assisted mobile application **Team:** Pavan · Prabhava · Prem · Pramath

---

Table of Contents

---

1. Introduction

The Smart Complaint Management System (SCMS) is a campus-oriented, cross-platform mobile application that digitises the entire lifecycle of a maintenance or facility complaint within an educational institution. Traditional complaint handling in colleges is largely manual — a student verbally reports an issue, or writes it in a register, after which the report is often lost, duplicated, or left unresolved with no accountability or tracking. SCMS replaces this process with a structured, transparent, and auditable digital workflow.

The system allows a student to raise a complaint from a mobile phone, attach photographic evidence that is automatically stamped with GPS coordinates and a timestamp, and track the complaint as it moves through a well-defined approval and resolution pipeline. The application integrates Artificial Intelligence, using Google's Gemini models, to improve the quality of complaints (grammar correction), reduce manual effort (automatic categorization), and prevent redundant work (duplicate detection through semantic similarity).

SCMS is engineered as three independent but cooperating services: a Flutter mobile application, a Node.js REST API backend, and a Python AI microservice. This separation reflects modern software engineering practice and allows each part of the system to be developed, tested, scaled, and maintained independently.

---

2. Problem Statement

Educational campuses generate a continuous stream of infrastructure and facility complaints — electrical faults, plumbing leaks, broken furniture, network issues, cleanliness problems, and more. The conventional handling of these complaints suffers from several well-defined problems:

- **No single point of submission.** Complaints are raised verbally, on paper, or through informal channels, making them easy to lose.
- **Lack of accountability.** There is no record of who reported an issue, who was assigned to fix it, or whether it was actually resolved.
- **No verifiable evidence.** Reports are descriptive only; there is no proof of the problem or of its resolution, allowing disputes and false closures.
- **Duplication of effort.** The same issue is frequently reported by many people, causing repeated inspections and wasted staff time.
- **Poor communication.** The complainant is rarely informed of progress, leading to frustration and repeated follow-ups.
- **No performance measurement.** Management has no data on complaint volumes, resolution times, or departmental performance.

Problem statement: *To design and develop a smart, mobile-first complaint management system that provides a single, accountable, and transparent channel for raising, tracking, and*

resolving campus complaints, strengthened by AI assistance for quality improvement and duplicate prevention, and by verifiable photographic evidence for both the problem and its resolution.

---

3. Objectives

The project was undertaken with the following objectives:

1. To provide a mobile application through which students can submit complaints quickly, with photographic evidence.
2. To automatically attach trustworthy metadata (GPS location and timestamp) to every photograph, preventing falsification.
3. To use AI to correct grammar in complaint descriptions, so complaints are clear and professional.
4. To use AI to automatically categorize complaints and assess severity, reducing manual triage.
5. To detect duplicate complaints using semantic similarity, so the same issue is not processed multiple times.
6. To implement a multi-role approval and resolution workflow with clearly defined responsibilities.
7. To require photographic proof of resolution before a complaint can be closed, ensuring genuine completion.
8. To notify all relevant parties in real time at every stage using push notifications.
9. To track Service Level Agreement (SLA) deadlines and flag breaches automatically.
10. To provide analytics and reporting (including Excel export) for management oversight.
11. To enforce secure, institution-restricted authentication using Google OAuth 2.0.

---

4. Scope of the Project

In scope:

- Android mobile application (built with a codebase that is portable to iOS).
- Four user roles: Student, Student Representative (SR), Staff, and Administrator.
- Complete complaint lifecycle from submission to closure, including an administrative verification loop.
- AI-assisted grammar correction, categorization, and duplicate detection.
- Photographic evidence with GPS and timestamp watermarking.
- Real-time push notifications.
- SLA tracking and automatic breach detection.

- Analytics dashboards and Excel export.
- Offline draft saving for complaints.

Out of scope (current version):

- Public/anonymous complaints (all users must authenticate with an institutional Google account).
- Payment or procurement workflows.
- Integration with external ticketing systems (e.g. ServiceNow).
- iOS App Store deployment (the codebase supports iOS, but only Android was built and tested).
- Web dashboard (management functions are performed from within the mobile app).

---

5. Existing System and Proposed System

5.1 Existing System

In most campuses the complaint process is manual or, at best, handled through a shared email inbox or a paper register. This has the following drawbacks: complaints are untracked, evidence is absent, duplicates are common, the complainant is not kept informed, and no performance data is captured. Where basic digital forms exist, they typically lack workflow, role separation, evidence handling, and any form of intelligence.

5.2 Proposed System

SCMS proposes a structured digital system with the following distinguishing characteristics:

- A **single mobile channel** for submission and tracking.
- A **formal, multi-stage workflow** with role-based responsibilities and an administrative verification step.
- **Evidence integrity** through GPS and timestamp watermarking of photos, and mandatory proof-of-resolution photos.
- **AI augmentation** for grammar, categorization, and duplicate detection.
- **Real-time notifications** and **SLA monitoring**.
- **Analytics** for institutional decision-making.

The proposed system converts an opaque, manual process into a transparent, accountable, and data-driven one.

---

6. System Requirements

6.1 Functional Requirements

- The system shall allow a user to sign in only with an institutional Google account (`rvce.edu.in`).
- The system shall allow a student to submit a complaint with a title, description, location, category, severity, tags, and one or more photographs.
- The system shall watermark each photograph with GPS coordinates and a timestamp at the time of capture.
- The system shall offer AI grammar correction and AI categorization suggestions during submission.
- The system shall warn the user if a similar complaint already exists.
- The system shall route each complaint through the defined approval and resolution workflow.
- The system shall require staff to upload proof photographs before a complaint can be marked resolved.
- The system shall require administrative verification before a complaint is completed.
- The system shall allow the complainant to rate a completed complaint.
- The system shall send push notifications on every relevant state change.
- The system shall track SLA deadlines and mark breaches.
- The system shall provide analytics and an Excel export to administrators.

6.2 Non-Functional Requirements

- **Security:** Institution-restricted authentication, JWT-based sessions, role-based access control, encrypted token storage.
- **Reliability:** The AI service is best-effort; its failure must not prevent complaint submission.
- **Performance:** REST API responses for standard operations should complete within typical mobile-network latency; AI calls are debounced to avoid excessive requests.
- **Usability:** Clean, consistent user interface following iOS design conventions; offline draft support.
- **Maintainability:** Clear separation of concerns across three services and, within the app, a layered clean architecture.
- **Portability:** Single Flutter codebase targeting Android and iOS.

6.3 Hardware and Software Requirements

Development environment:

- A development machine (Windows/macOS/Linux) with at least 8 GB RAM.
- Flutter SDK (Dart >= 3.4), Android Studio / VS Code, Android SDK.

- Node.js (18+) and npm.
- Python 3.11+ and pip.
- PostgreSQL 16 with the pgvector extension (provided via the `pgvector/pgvector:pg16` Docker image).
- A Google Cloud OAuth client and a Google AI Studio (Gemini) API key.
- A Firebase project for Cloud Messaging.

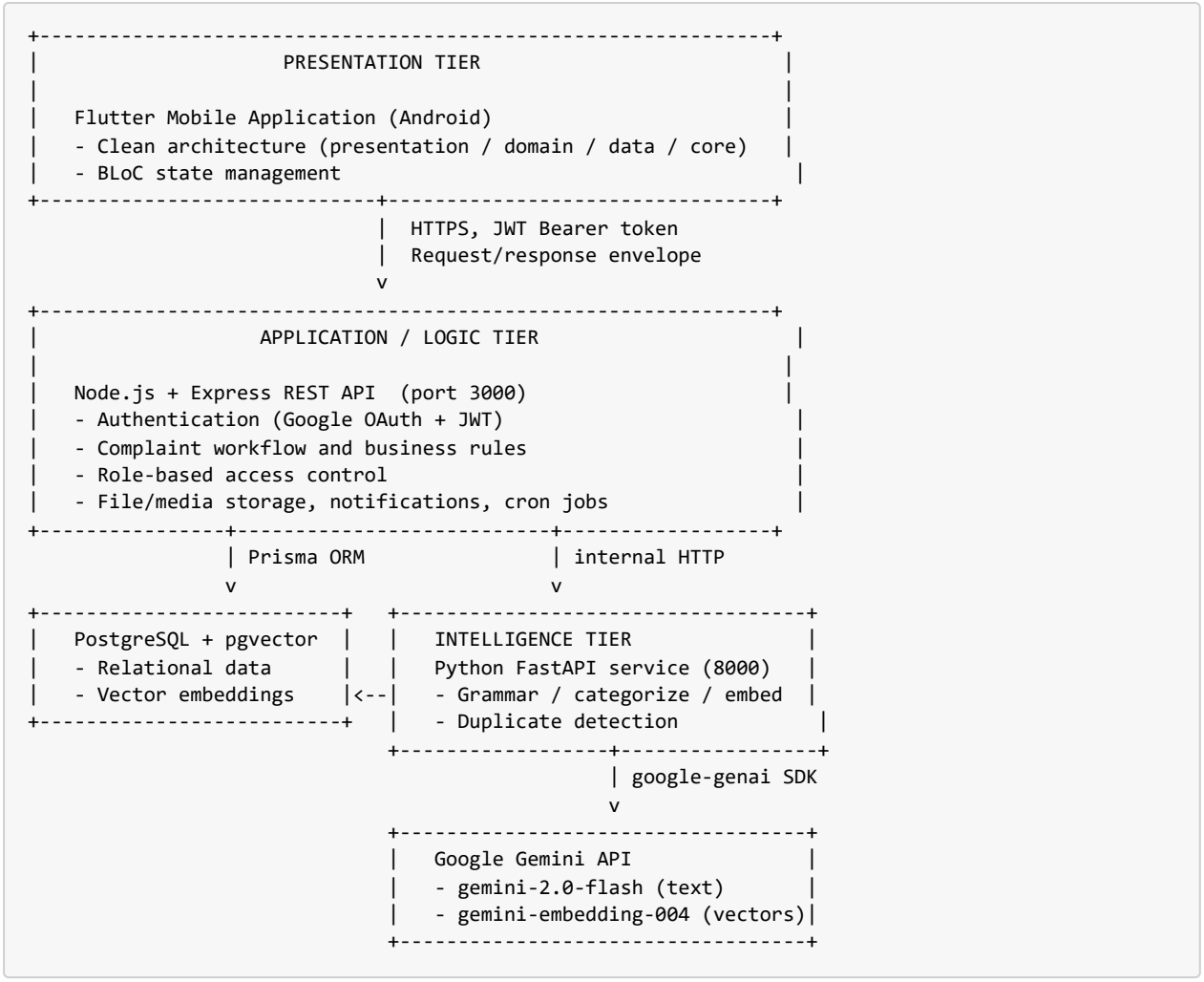
Runtime (client): An Android device or emulator running Android 5.0 (API 21) or higher, with camera, GPS, and internet connectivity.

---

7. System Architecture

7.1 Architectural Overview

SCMS follows a **three-tier, service-oriented architecture**. The presentation tier is the Flutter mobile application. The application/logic tier is the Node.js backend, which owns all business rules, authentication, and data. The intelligence tier is the Python AI microservice, which encapsulates all interaction with Google Gemini and with the vector database features. A single PostgreSQL database (with the pgvector extension) serves as the persistent store.



7.2 Key Architectural Principles

1. **The mobile app never contacts the AI service directly.** All AI requests pass through the Node.js backend (`/api/ai/*`), which forwards them to the Python service through a proxy. This maintains a single authentication and security boundary.
2. **The AI service is a best-effort augmentation, not a hard dependency.** Every AI endpoint returns a safe default on failure, and the backend proxy wraps every AI call in error handling. If Gemini or the AI service is unavailable, the complaint system continues to function normally.
3. **A uniform response envelope.** Every backend response is wrapped as `{ "success": true, "data": ... }` or `{ "success": false, "error": ... }` . The mobile client strips this envelope in a network interceptor, so data-handling code always receives a clean payload.
4. **One database for both relational and vector data.** PostgreSQL with pgvector stores structured complaint data and the AI-generated embeddings together, eliminating the need for a separate vector database.

7.3 Request Flow Example — Submitting a Complaint

```
Student fills form
   -> (while typing) app debounces 800ms, calls /api/ai/grammar-check and /api/ai/categorize
   -> backend proxies to Python service -> Gemini -> suggestions returned
   -> app shows grammar and category suggestions, and a duplicate warning if applicable
Student captures photo
   -> CameraService captures -> LocationService gets GPS
   -> WatermarkService stamps GPS + timestamp onto the image
Student submits
   -> multipart POST /api/complaints (fields + media files)
   -> backend validates, creates Complaint row, stores media, resolves department,
      generates a human-readable complaint number, sets SLA deadline
   -> backend calls /embed on the AI service to store the description vector
   -> backend notifies the relevant Student Representative via FCM
   -> response returned; app updates the student's complaint list
```

---

8. Technology Stack and Justification

The following table lists every major technology choice, the reason it was selected, and the principal alternatives that were considered and rejected.

Layer	Technology	Justification	Alternatives rejected
Mobile application	Flutter (Dart)	Single codebase for Android and iOS; compiles to native ARM for good performance; rich widget system enabled a polished, consistent UI; hot reload accelerated development.	React Native (JavaScript bridge is slower, UI less consistent); native Android/Kotlin (would require a separate iOS codebase).
Mobile state management	BLoC / Cubit (flutter_bloc)	Enforces a clear separation between UI and business logic through an event-to-state flow; predictable and testable; the standard for large Flutter applications.	setState (does not scale, mixes logic into widgets); Provider/Riverpod (viable, but BLoC's structure suited the workflow-heavy application better).
Backend runtime	Node.js with Express	Non-blocking, event-driven I/O is ideal for an API that is predominantly network- and database-bound; very large package ecosystem; rapid development; JavaScript aligns naturally with JSON handling.	Django/Flask (Python was reserved for the AI service to keep concerns separate); Spring Boot (heavier and slower to iterate for a student project).
AI microservice	Python with FastAPI	Python has the strongest AI/ML ecosystem, including Google's official SDK and NumPy for vector mathematics; FastAPI is asynchronous, high-performance, and auto-generates interactive API documentation; Pydantic provides request/response validation.	Implementing AI inside Node.js (poor ML tooling); Flask (synchronous by default, lacks FastAPI's built-in validation).
AI models	Google Gemini (gemini-2.0-flash, gemini-embedding-004)	Generous free tier suitable for an academic project; strong performance on grammar and classification tasks; a dedicated embedding model for semantic search; a simple, well-documented SDK.	OpenAI GPT (paid, requires billing); self-hosted large language models (too resource-intensive for available hardware).
Database	PostgreSQL	The data is highly relational (users, complaints, departments, categories) and benefits from ACID guarantees; the pgvector extension allows AI embeddings and similarity search to reside in the same database as the relational data.	MongoDB (weak relational support, no native vector search at the time); MySQL (no first-class vector extension comparable to pgvector).

Layer	Technology	Justification	Alternatives rejected
Object-relational mapping	Prisma	Type-safe database access, an auto-generated client, a clean and readable schema definition, and a robust migration system; reduces the risk of SQL errors and speeds development.	Raw SQL (error-prone, no type safety); Sequelize (older, less ergonomic API).
Authentication	Google OAuth 2.0 with JWT	The campus already uses Google Workspace, so students authenticate with existing credentials and sign-ups are automatically restricted to the institutional domain; JWTs provide stateless, scalable sessions.	Email/password (password-storage risk, no domain restriction, additional friction); server-side sessions (stateful, harder to scale across services).
Vector search	pgvector	Enables cosine-similarity search directly within PostgreSQL, avoiding a second datastore.	Pinecone / Weaviate (external, paid, additional infrastructure).
Push notifications	Firebase Cloud Messaging	Free, reliable, and the de facto standard for Android push notifications; integrates directly with the Flutter Firebase SDK.	A custom WebSocket server (would require building delivery, retry, and scaling ourselves).
Offline storage	Hive	A lightweight, fast, pure-Dart store well suited to saving complaint drafts offline.	SQLite (unnecessarily heavy for simple drafts); SharedPreferences (not designed for structured objects).
Media upload	Multer with local storage	Straightforward multipart handling; local disk storage is adequate at campus scale.	Cloud object storage such as AWS S3 (added cost and complexity beyond project scope).

8.1 Full Dependency Summary

Flutter (client): dio, flutter_bloc, go_router, flutter_secure_storage, google_sign_in, image_picker, camera, geolocator, geocoding, image, firebase_core, firebase_messaging, flutter_local_notifications, cached_network_image, fl_chart, hive / hive_flutter, connectivity_plus, permission_handler, flutter_dotenv, diff_match_patch, path_provider, google_fonts, intl.

Node.js backend: express, @prisma/client / prisma, google-auth-library, jsonwebtoken, firebase-admin, multer, exceljs, node-cron, helmet, cors, morgan, axios, zod, dotenv, uuid.

Python AI service: fastapi, uvicorn, google-genai, psycopg2-binary, pgvector, numpy, pydantic, httpx, python-dotenv.

---

9. Database Design

The database is a PostgreSQL schema managed through Prisma. It consists of nine tables. The central entity is `Complaint`, which is related to `User`, `MediaItem`, and `ComplaintUpdate`, and references `Category`, `Department`, `Zone`, and `Tag` values.

9.1 Entity Overview

Table	Purpose
`users`	Registered users, their role, department/zone, and FCM token.
`complaints`	The core complaint record, including status, AI flags, SLA data, and rating.
`media_items`	Photographs and videos attached to complaints, with GPS and watermark metadata; each is marked ORIGINAL or PROOF.
`complaint_updates`	An append-only timeline of every status change (who, from, to, when, notes).
`departments`	Departments responsible for resolving complaints.
`categories`	Complaint categories, each mapped to a default department.
`zones`	Physical campus zones/locations.
`tags`	Predefined labels that can be attached to complaints.
`allowed_domains`	The allow-list of email domains permitted to sign up.
`refresh_tokens`	Persisted refresh tokens for session renewal.

9.2 Principal Fields of the Complaint Entity

The `complaints` table captures the full state of a complaint:

- **Identity and content:** `id`, `complaintNumber` (human-readable, e.g. `SCMS-2026-00007`), `title`, `description`, `location`.
- **Geolocation:** `gpsLatitude`, `gpsLongitude`, `gpsPlaceName`.
- **Classification:** `categoryId`, `departmentId`, `severity`, `tags`.
- **Workflow state:** `status`, `submittedById`, `assignedToId`, `reviewedBySrId`, `srRejectionCause`.
- **AI metadata:** `isGrammarCorrected`, `isAiCategorized`, `aiConfidenceScore`, `duplicateGroupId`.
- **SLA:** `slaDeadline`, `isSlaBreached`.
- **Outcome:** `rating`, `ratingComment`, `resolvedAt`, `completedAt`.
- **Auditing:** `createdAt`, `updatedAt`.

The description's vector embedding is stored in an additional `embedding` column (of pgvector type), provisioned by the AI service at startup and populated after each complaint is created.

9.3 Entity-Relationship Summary

```
users (1) ----------< (many) complaints        [a user submits many complaints]
complaints (1) -----< (many) media_items        [ORIGINAL and PROOF media]
complaints (1) -----< (many) complaint_updates  [status-change timeline]
categories (1) -----> (1) departments           [default routing department]
complaints (many) --> (1) categories            [classification]
complaints (many) --> (1) departments           [responsible department]
```

---

10. Module Description

The system is organised into three services and, within the mobile application, into role-based feature modules.

10.1 Python AI Service (`scms_ai_service/`)

This FastAPI microservice encapsulates all artificial intelligence functionality. It exposes four endpoints, each of which is designed to fail safe (returning a benign default rather than an error).

- **`main.py`** — the FastAPI application. It mounts the four routers, exposes a `GET /health` probe, configures CORS, and on startup guarantees that the pgvector `embedding` column exists.
- **`routers/grammar.py`** — `POST /grammar-check`. Submits the complaint text to Gemini and returns corrected text along with word-level differences (marked EQUAL, DELETE, or INSERT) so the app can visually highlight the changes.
- **`routers/categorize.py`** — `POST /categorize`. Uses Gemini in structured-JSON mode to return a category, a severity level, and a confidence score.
- **`routers/embed.py`** — `POST /embed`. Generates a 768-dimensional embedding of the complaint description and stores it in PostgreSQL. It is invoked by the backend after the complaint record exists.
- **`routers/duplicate.py`** — `POST /check-duplicate`. Performs a cosine-similarity search over stored embeddings and returns matches above a configured threshold.
- **`services/gemini_client.py`** — a single wrapper around the google-genai SDK used by all routers, with try/except fallbacks throughout.
- **`services/db_client.py`** — a psycopg2 connection pool with pgvector support, providing embedding storage and similarity queries.
- **`models/schemas.py`** — Pydantic request and response models for all endpoints.

10.2 Node.js Backend (`scms_backend/`)

The backend is the authoritative core of the system. It owns authentication, all business rules, the database, media storage, notifications, and scheduled jobs.

- `app.js` / `server.js` — configure Express with Helmet (security headers), CORS, and Morgan (logging); mount all routers under `/api`; serve uploaded media statically; and register the global error handler.
- **Routes (`routes/`):** one router per resource — `auth`, `complaints`, `sr`, `analytics`, `ai`, `departments`, `categories`, `tags`, `zones`, and `users`.
- **Middleware (`middleware/`):** `authenticate.js` (JWT verification), `requireRole.js` (role-based access control), `upload.js` (Multer media handling), `validateBody.js` (schema validation), and `errorHandler.js`.
- **Services (`services/`):** `aiProxy.js` (forwards requests to the Python service with safe fallbacks), `googleAuth.js` (verifies Google ID tokens), `fcm.js` (Firebase push notifications), `storage.js` (media persistence), and `complaintNumber.js` (human-readable ID generation).
- **Scheduled jobs (`jobs/`):** `slaScheduler.js` marks complaints whose SLA deadline has passed as breached and notifies administrators; `srAutoApprove.js` auto-approves complaints that have remained in SR review beyond a threshold.
- **Utilities (`utils/`):** `enrichComplaints.js` (joins related data so the client receives display-ready fields), `jwtHelper.js`, `responseHelper.js` (the success/error envelope), and `logger.js`.

10.3 Flutter Application (`scms_flutter/`)

The mobile application is organised using a clean, layered architecture:

- `core/` — cross-cutting foundations: theme and design system, constants (API routes, SLA limits), the Dio HTTP client with authentication and envelope-stripping interceptors, error types, and utilities including the watermark painter.
- `data/` — models (JSON serialization), remote data sources (per-resource Dio calls), local data sources (Hive drafts and secure token storage), and repositories that combine remote and local sources with offline fallback.
- `domain/` — entities and single-responsibility use cases (login, submit complaint, get my complaints, get analytics, SR approve/reject, update status).
- `presentation/` — BLoCs/Cubits (auth, complaint, submit_complaint, sr_review, analytics, all_complaints), reusable widgets, and screens organised by role.

The application's screens are grouped into the following role-based feature modules.

10.3.1 Student Module

Screens: splash, onboarding, login, home dashboard, submit complaint, my complaints, complaint detail, duplicate complaints, and rating. The submit-complaint screen is the centrepiece: while the student types, a Cubit debounces input by 800 milliseconds and requests grammar correction and category suggestions; it also warns of possible duplicates. Photographs are captured, GPS-tagged, and watermarked before submission.

10.3.2 Student Representative (SR) Module

Screens: SR dashboard (a queue of complaints pending review, with severity filters) and SR review detail (approve, or reject with a stated reason). The SR acts as the first line of validation, filtering out invalid or spam complaints before they enter the resolution pipeline.

10.3.3 Staff Module

Screens: staff dashboard (assigned tasks with live counts) and staff complaint detail. Staff members begin work on a complaint and, to mark it resolved, must submit proof-of-resolution photographs together with notes. A complaint cannot be resolved without this evidence.

10.3.4 Administrator Module

Screens: admin dashboard (analytics — key indicators, department and category charts, recent SLA breaches), a system-wide complaints list, assignment of complaints to staff, verification of resolutions (approve or send back for rework), and Excel export of filtered complaint data.

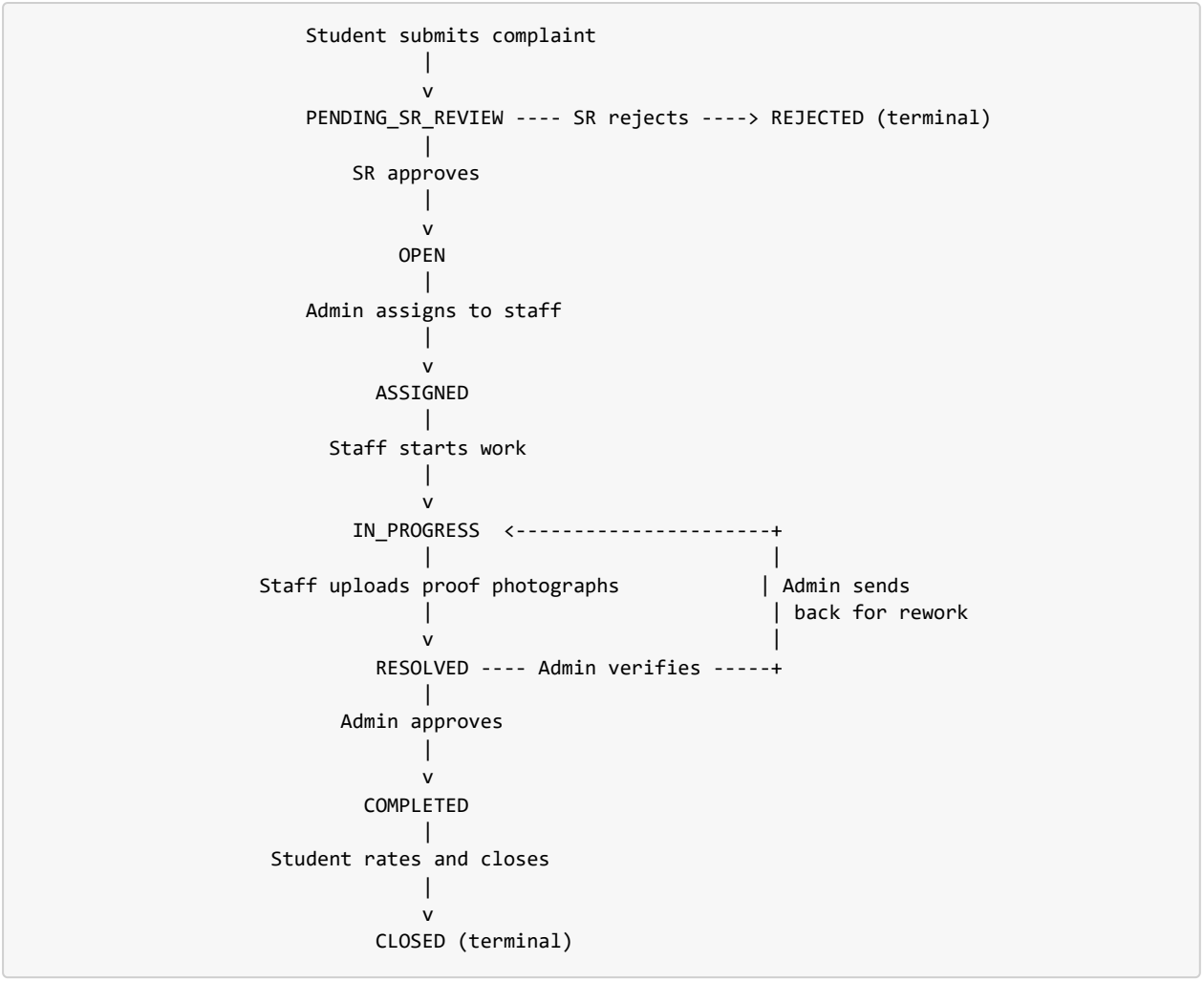
10.3.5 Shared Module

A common bottom-navigation shell hosts a role-specific dashboard alongside shared "All Complaints", "Statistics", and "Profile" tabs. Settings (theme selection and notification preferences) and the profile screen are common to all roles.

---

11. Complaint Lifecycle and Workflow

The complaint lifecycle is a formal state machine consisting of eight statuses. It embeds an administrative verification loop that ensures resolutions are genuine.



Stage descriptions:

1. **PENDING_SR_REVIEW** — the initial state on submission. The relevant Student Representative reviews the complaint.
2. **OPEN** — the SR has approved the complaint as valid; it awaits assignment. (If the SR rejects it, it moves to **REJECTED**, a terminal state, with a recorded reason.)
3. **ASSIGNED** — an administrator has assigned the complaint to a specific staff member.
4. **IN_PROGRESS** — the staff member has begun work.
5. **RESOLVED** — the staff member has uploaded proof-of-resolution photographs; administrators are notified. Resolution without proof is rejected by the server.
6. **COMPLETED** — an administrator has verified the resolution and approved it; the complainant is notified to rate. Alternatively, the administrator may send the complaint back for rework, returning it to **IN_PROGRESS** with the same staff member re-notified.
7. **CLOSED** — the complainant has rated the completed complaint, closing it (terminal).

Throughout this lifecycle, every transition is recorded in the `complaint_updates` timeline, a push notification is dispatched to the appropriate party, and the SLA deadline is monitored by a background job.

---

12. User Roles and Permissions

The system defines four roles, encoded in JWTs using the convention `ROLE_USER`, `ROLE_SR`, `ROLE_STAFF`, and `ROLE_ADMIN`. Access is enforced by role-checking middleware on the backend and by role-based routing and redirects in the mobile application.

Capability	Student	SR	Staff	Admin
Submit a complaint	Yes	Yes	Yes	Yes
View own complaints	Yes	Yes	Yes	Yes
View all complaints (read-only)	Yes	Yes	Yes	Yes
Approve / reject a pending complaint	No	Yes	No	No
Assign a complaint to staff	No	No	No	Yes
Mark in progress / upload proof of resolution	No	No	Yes (assigned)	No
Verify resolution (approve / send back)	No	No	No	Yes
Rate a completed complaint	Yes (own)	Yes (own)	Yes (own)	Yes (own)
View analytics	Yes	Yes	Yes	Yes
Export complaints to Excel	No	No	No	Yes

Read access to complaints and analytics is intentionally open to all authenticated users to promote transparency; all write operations remain strictly guarded by role.

---

13. API Design

All endpoints are served under the `/api` prefix and, with the exception of authentication endpoints, require a valid JWT Bearer token. Every response uses the standard success/error envelope.

13.1 Authentication

Method	Endpoint	Description
POST	`/api/auth/google`	Verify a Google ID token and issue access and refresh JWTs.
POST	`/api/auth/refresh`	Exchange a refresh token for a new access token.
GET	`/api/auth/me`	Return the authenticated user's profile.
POST	`/api/auth/logout`	Invalidate the session.
GET	`/api/auth/allowed-domains`	List permitted sign-up domains.

13.2 Complaints

Method	Endpoint	Description
GET	`/api/complaints`	List complaints (supports scope, status, severity, and text filters).
GET	`/api/complaints/my`	List the current user's complaints.
POST	`/api/complaints`	Create a complaint (multipart, with media).
GET	`/api/complaints/:id`	Retrieve a single complaint.
PATCH	`/api/complaints/:id`	Owner edit of a complaint.
DELETE	`/api/complaints/:id`	Owner deletion of a complaint.
PATCH	`/api/complaints/:id/status`	Update workflow status.
PATCH	`/api/complaints/:id/assign`	Assign to a staff member (admin).
POST	`/api/complaints/:id/resolve`	Staff resolution with proof media.
POST	`/api/complaints/:id/verify-resolution`	Admin verification (approve or rework).
POST	`/api/complaints/:id/rating`	Complainant rating.
GET	`/api/complaints/export`	Excel export (admin).

13.3 Student Representative

Method	Endpoint	Description
GET	`/api/sr/pending`	List complaints awaiting review.
POST	`/api/sr/:id/approve`	Approve a complaint.
POST	`/api/sr/:id/reject`	Reject a complaint with a reason.

13.4 Artificial Intelligence (Proxy)

Method	Endpoint	Description
POST	`/api/ai/grammar-check`	Grammar correction with diffs.
POST	`/api/ai/categorize`	Category and severity suggestion.
POST	`/api/ai/check-duplicate`	Duplicate similarity check.

13.5 Reference Data and Analytics

Method	Endpoint	Description
GET	`/api/departments`, `/api/categories`, `/api/tags`, `/api/zones`	Reference data for form population.
GET	`/api/analytics/summary`	Aggregate dashboard statistics.
GET	`/api/analytics/by-department`, `/api/analytics/by-category`, `/api/analytics/sla-breaches`	Detailed analytics.
GET	`/api/users`	Staff listing for assignment (admin).
POST	`/api/users/fcm-token`	Register a device push token.

---

14. Artificial Intelligence Subsystem

The AI subsystem provides three user-facing capabilities, all delivered through the Python microservice and Google Gemini.

14.1 Grammar Correction

The complaint description is submitted to Gemini, which returns a corrected version. The service computes word-level differences between the original and the correction and returns them as a sequence of EQUAL, DELETE, and INSERT operations. The mobile application renders these differences so the student can review and accept the improved text.

14.2 Automatic Categorization

The description is sent to Gemini with instructions to respond in a fixed JSON structure containing a category, a severity level, and a confidence score. The application presents this

suggestion, which the student may accept or override. Accepting it reduces manual triage effort and improves routing accuracy.

14.3 Duplicate Detection

Duplicate detection is based on **semantic similarity** rather than exact text matching. The process is as follows:

1. When a complaint is created, its description is converted into a 768-dimensional vector (an embedding) using Gemini's embedding model. This vector numerically represents the meaning of the text.
2. The vector is stored in PostgreSQL in a pgvector column.
3. When a new complaint is being written, its provisional vector is compared against existing vectors using **cosine similarity**, a measure of the angle between two vectors that ranges from 0 (unrelated) to 1 (identical in meaning).
4. Complaints whose similarity exceeds a configured threshold are returned as potential duplicates.

Because this method operates on meaning rather than exact wording, it correctly identifies duplicates even when they are phrased differently — for example, "the tube light is not working" and "the light in the corridor is broken".

14.4 Fail-Safe Design

Every AI endpoint is wrapped in exception handling and returns a benign default on any failure: grammar correction returns the original text unchanged, categorization returns a neutral default, and duplicate detection returns an empty result. The backend proxy adds a second layer of the same protection. Consequently, an outage of the AI service or of Gemini never prevents a complaint from being submitted.

---

15. Security Design

- **Authentication:** Sign-in is exclusively through Google OAuth 2.0, restricted to the institutional domain `rvce.edu.in`. There is no password-based login, eliminating password-storage risk.
- **Token verification:** Google ID tokens are verified server-side using the official Google authentication library before any session is created.
- **Session management:** The backend issues its own short-lived access JWT and a longer-lived refresh JWT. The mobile client transparently refreshes the access token on expiry.
- **Secure storage:** Tokens are stored on the device using encrypted secure storage.

- **Authorization:** Role-based access control middleware guards every state-changing endpoint; read access is deliberately broad, write access is narrow.
- **Transport and headers:** The backend applies Helmet security headers and CORS restrictions, and communicates with clients over HTTPS.
- **Data minimisation:** Sensitive personal fields (for example, submitter email in analytics feeds) are excluded from broad read responses to avoid unnecessary exposure.
- **Evidence integrity:** GPS and timestamp watermarks are applied to photographs at capture, and proof-of-resolution photographs are mandatory, reducing the possibility of falsified reports or closures.

---

16. Feature Summary

Feature	Description
Institution-restricted login	Google OAuth 2.0 limited to the campus domain.
Complaint submission with media	Title, description, location, category, severity, tags, and photographs.
GPS and timestamp watermarking	Location and time stamped onto each photograph at capture.
AI grammar correction	Gemini-based correction with visual diff review.
AI categorization	Automatic category and severity suggestion.
Duplicate detection	Semantic similarity search using vector embeddings.
Multi-role workflow	Eight-stage lifecycle with SR approval and admin verification.
Proof of resolution	Mandatory photographic evidence before closure.
Real-time notifications	Firebase push notifications at every stage.
SLA tracking	Automatic deadline monitoring and breach flagging.
Analytics dashboards	Key indicators and department/category charts.
Excel export	Administrator export of filtered complaint data.
Ratings	Complainant rating of completed complaints.
Offline drafts	Local saving of unsent complaints.
Role-based navigation	Each role has a tailored dashboard within a shared shell.

---

17. Team Structure and Work Division

The project was developed by a four-member team using a strict file-ownership model: each member owns a defined set of files and does not modify another member's files directly. Work is organised across separate Git branches, integrated through pull requests, with progress tracked in a shared context document.

Member	Responsibility	Contribution
Pavan (Project Lead)	Flutter — foundation, core, data layer, student flow, and integration	Scaffolded the entire monorepo; built the design system, networking layer, data models, repositories, and shared widgets on which the rest of the app depends; implemented all student-facing screens; built the camera, location, and GPS/timestamp watermarking pipeline and the AI-assistance widgets (grammar and duplicate banners); configured routing and application-wide dependency injection; performed final integration of all members' work.
Prabhava	Flutter — Staff, SR, Admin, Settings, and Notifications	Built the role dashboards and detail screens for Staff, Student Representative, and Administrator; implemented the SR approval/rejection flow, the staff proof-of-resolution submission, and the administrator analytics dashboard and charts; built the complete notification system (Firebase Cloud Messaging, local notifications, in-app banners with deep-linking) and the settings screen.
Prem	Node.js Backend (entire service)	Implemented the complete REST API, including Google OAuth with JWT authentication, all complaint CRUD and workflow endpoints (resolve, verify-resolution, assign), the SR and analytics routes, the AI proxy, media upload handling, the Firebase push-notification service, the Prisma schema and migrations, database seeding, the SLA and auto-approval scheduled jobs, and the Excel export.
Pramath	Python AI Service (entire service)	Implemented the FastAPI microservice and all four Gemini-powered endpoints — grammar correction with word-level diffs, structured categorization, embedding generation, and duplicate detection via pgvector cosine similarity — together with the fail-safe design that guarantees AI outages never disrupt the core system.

Collaboration practices: a monorepo with per-member branches; no direct commits to the main branch (all changes via pull request); a single source-of-truth progress document; and a defined ownership map for shared configuration files, edited only by the project lead.

---

18. Testing and Verification

- **Backend:** Endpoints were verified directly using API clients and by exercising the running application against a seeded database. Route files are syntax-checked, and an end-to-end

workflow smoke test validates assignment, proof-based resolution, admin verification, the rework loop, rejection, and Excel export.

- **AI service:** Each endpoint was tested independently using command-line requests, confirming both correct output and graceful fallback behaviour on failure.
- **Mobile application:** Static analysis (`flutter analyze`) is maintained at a clean state ("No issues found"), and debug builds are validated. The application was exercised on-device across the full workflow in both light and dark themes.
- **Demonstration data:** A seed script populates a representative dataset — an administrator, three Student Representatives, four staff members, five students, and twenty complaints distributed across every stage of the lifecycle with realistic timelines and proof media — enabling reproducible demonstrations.

---

19. Deployment and Setup

The three services are configured to run locally for development and demonstration.

PostgreSQL (with pgvector): started from the project root using Docker Compose (`docker-compose up postgres -d`), which uses the `pgvector/pgvector:pg16` image.

Node.js backend:

```
cd scms_backend
npm install
cp .env.example .env          # set GOOGLE_CLIENT_ID, Firebase credentials, database URL
npx prisma migrate dev        # apply schema
node prisma/seed.js           # seed reference data
node prisma/seed_sample_data.js  # seed demonstration users and complaints
npm run dev                   # start on port 3000
```

Python AI service:

```
cd scms_ai_service
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env          # set GEMINI_API_KEY
uvicorn main:app --reload --port 8000
```

Flutter application:

```
cd scms_flutter
flutter pub get
flutter run                   # on a connected device or emulator
```

For demonstration, a development-only sign-in path allows selecting a seeded user of a specific role without completing the full Google OAuth flow; this path is available only when the backend runs in development mode.

---

20. Limitations and Future Enhancements

Current limitations:

- The application is built and tested for Android only, although the codebase is iOS-ready.
- Media is stored on the local server disk rather than in cloud object storage.
- User preferences such as theme are held in memory and reset on a cold start.
- There is no dedicated web dashboard; management functions are performed within the mobile app.

Planned enhancements:

- Cloud object storage for media and horizontal scaling of the backend.
- A web-based administrative dashboard.
- Persistent user preferences and richer personalisation.
- Expanded analytics, including per-day trend series and predictive SLA-risk indicators.
- Support for anonymous or guest complaints with appropriate safeguards.
- iOS release.

---

21. Conclusion

The Smart Complaint Management System demonstrates a complete, modern, full-stack application built to a professional standard. It addresses a genuine institutional problem — the opaque and unaccountable handling of campus complaints — with a solution that is transparent, accountable, and data-driven. The system combines a polished mobile client, a robust and secure backend, and an AI microservice that meaningfully improves complaint quality and eliminates redundant work, all integrated through a carefully designed service-oriented architecture.

Beyond its functional achievements, the project reflects sound engineering discipline: a clean layered architecture, strict separation of concerns across services, a formal and auditable workflow, defensive and fail-safe integration with external AI, and a well-structured collaborative development process across a four-member team. The result is a system that is not only feature-complete but also maintainable, extensible, and ready for real deployment.

---

22. Appendix A: Project Directory Structure

The project is organised as a monorepo containing the three services and shared documentation. The following annotated trees describe where each part of the system resides and what it does.

22.1 Top-Level Layout

```
SCMS/
├── scms_flutter/        Flutter mobile application (presentation tier)
├── scms_backend/        Node.js + Express REST API (application tier)
├── scms_ai_service/     Python FastAPI AI microservice (intelligence tier)
├── docs/                Reports, documentation, and screenshots
├── scripts/             Development helper scripts (start services, tunnels)
├── docker-compose.yml   PostgreSQL (pgvector) and service orchestration
├── SCMS_PRD.md          Full product requirements specification
├── TEAM_WORKDIVISION.md File-ownership map for the four-member team
├── CONTEXT.md           Single source of truth for project state and history
└── CLAUDE.md            Repository guidance and conventions
```

22.2 Flutter Application (`scms_flutter/lib/`)

The application follows a clean, layered architecture. Files are grouped by layer (core, data, domain) and, within the presentation layer, by feature and role.

```
lib/
├── main.dart                    Application entry point; Firebase init, dependency
│                                injection (repositories and BLoCs), notification setup
├── app.dart                     Root widget; go_router configuration with role-based
│                                redirects; binds live theme preferences
├── firebase_options.dart        Generated Firebase configuration
│
├── core/                        Cross-cutting foundations (owned by the whole app)
│   ├── app_preferences.dart     In-memory theme and notification preferences
│   ├── constants/               API routes, app limits, route names, predefined tags
│   ├── theme/                   Design system: colours, text styles, light/dark themes
│   ├── network/                 Dio HTTP client (auth + envelope interceptors),
│   │                            connectivity checks, server-URL override
│   ├── errors/                  Exception and failure type hierarchies
│   └── utils/                   Date formatting, validators, extensions, logger,
│                                category icons, GPS/timestamp watermark painter
│
├── data/                        Data layer (models, sources, repositories)
│   ├── models/                  JSON (de)serialization for User, Complaint, Category,
│   │                            Department, Analytics, Grammar, Duplicate, Rating, SR
│   ├── datasources/remote/      Per-resource Dio API calls (auth, complaint, sr_review)
│   ├── datasources/local/       Hive drafts and encrypted token storage
│   └── repositories/            Combine remote and local sources with offline fallback
│
├── domain/                      Business layer (framework-independent)
│   ├── entities/                Core domain objects (User, Complaint)
│   └── usecases/                Single-purpose actions (login, submit, get complaints,
│                                analytics, SR approve/reject, update status)
│
├── presentation/                UI layer
│   ├── bloc/                    State management, one folder per feature:
│   │                            auth, complaint, submit_complaint, sr_review,
│   │                            analytics, all_complaints
│   ├── widgets/
│   │   ├── common/              Reusable UI: buttons, text fields, chips, scaffolds,
│   │   │                        grouped list rows, segmented tabs, overlays
│   │   ├── complaint/           Complaint card, status badge, SLA timer, media capture,
│   │   │                        category/tag selectors, grammar and duplicate banners
│   │   ├── dashboard/           Dashboard hero, stat breakdowns, trend sparkline,
│   │   │                        attention card, quick actions, SR summary
│   │   ├── analytics/           Charts and stat cards
│   │   └── notification/        Notification badge
│   └── pages/                   Screens grouped by role/flow:
│       ├── splash, onboarding, auth      Entry and login (Google sign-in)
│       ├── home                          Student dashboard
│       ├── complaint                     Submit, my complaints, detail, rating, duplicates
│       ├── complaints                    Shared system-wide complaints feed
│       ├── staff                         Staff dashboard and task detail (proof upload)
│       ├── sr                            SR dashboard and review (approve/reject)
│       ├── admin                         Admin dashboard and complaints list
│       ├── stats, profile, settings      Shared analytics, profile, and settings
│       ├── shell/main_shell.dart         Bottom-navigation shell for all roles
│       └── route_helpers.dart            Role-specific route registration
│
└── services/                    Singletons not tied to a single BLoC:
    ├── notification_service.dart Firebase messaging, local notifications, in-app banners
    ├── camera_service.dart       Photo capture and gallery selection
    ├── location_service.dart     GPS acquisition and reverse geocoding
    ├── watermark_service.dart    Stamps GPS and timestamp onto photographs
    ├── grammar_service.dart      Standalone grammar-check client
    ├── storage_service.dart      Local complaint-draft folder management
    └── analytics_service.dart    Screen and event logging
```

22.3 Node.js Backend (`scms_backend/`)

```
scms_backend/
├── server.js                    Process entry point
├── package.json                 Dependencies and scripts
├── prisma/
│   ├── schema.prisma            Database schema (nine models)
│   ├── seed.js                  Seeds departments, categories, zones, tags, domains
│   └── seed_sample_data.js      Seeds demonstration users and complaints
└── src/
    ├── app.js                   Express setup: middleware, routers, static media,
    │                            error handler
    ├── routes/                  One router per resource:
    │   ├── auth.js              Google OAuth verification and JWT issuance
    │   ├── complaints.js        Complaint CRUD and full workflow endpoints
    │   ├── sr.js                SR approve/reject
    │   ├── ai.js                Proxy to the Python AI service
    │   ├── analytics.js         Aggregate statistics
    │   ├── departments.js, categories.js, tags.js, zones.js   Reference data
    │   └── users.js             User listing and FCM token registration
    ├── middleware/
    │   ├── authenticate.js      JWT verification (with dev-only mock path)
    │   ├── requireRole.js       Role-based access control
    │   ├── upload.js            Multer multipart media handling
    │   ├── validateBody.js      Request validation
    │   └── errorHandler.js      Centralised error handling
    ├── services/
    │   ├── aiProxy.js           Forwards AI requests with safe fallbacks
    │   ├── googleAuth.js        Verifies Google ID tokens
    │   ├── fcm.js               Firebase push notifications
    │   ├── storage.js           Media file persistence
    │   └── complaintNumber.js   Human-readable complaint ID generation
    ├── jobs/
    │   ├── slaScheduler.js      Cron job: flags SLA breaches, notifies admins
    │   └── srAutoApprove.js     Cron job: auto-approves stale SR reviews
    └── utils/
        ├── enrichComplaints.js  Joins related data into display-ready fields
        ├── jwtHelper.js         Token creation and parsing
        ├── responseHelper.js    Standard success/error response envelope
        └── logger.js            Logging utility
```

22.4 Python AI Service (`scms_ai_service/`)

```
scms_ai_service/
├── main.py                      FastAPI app; mounts routers, health probe,
│                                startup pgvector column provisioning
├── requirements.txt             Python dependencies
├── routers/
│   ├── grammar.py               POST /grammar-check (correction with word diffs)
│   ├── categorize.py            POST /categorize (category, severity, confidence)
│   ├── embed.py                 POST /embed (store 768-d description vector)
│   └── duplicate.py             POST /check-duplicate (cosine similarity search)
├── services/
│   ├── gemini_client.py         Single wrapper over the google-genai SDK
│   └── db_client.py             psycopg2 pool with pgvector; store and query vectors
└── models/
    └── schemas.py               Pydantic request/response models
```

End of document.